

week-09

Week 9. 멀티에이전트 개념: 설계 판단, 4대 패턴, 평가

강의 + 수업 설계 실습 + Homework 설계 과제 참조: Notion ch6 「멀티 에이전트 시스템」, Anthropic 「How we built our multi-agent research system」(2025), Cognition 「Don't build multi-agents」(2025)

학습 목표

- 단일 에이전트의 한계를 작업 구조 관점에서 설명한다
 - 멀티에이전트의 4대 패턴(계층형·협력형·순차형·시장형)을 구분한다
 - 핸드오프와 공유 메모리의 역할을 설명한다
 - 멀티에이전트가 “필요한 작업”과 “쓰지 말아야 할 작업”을 자신의 사례로 판단한다
 - 자기 도메인의 작업을 멀티에이전트 설계안으로 구체화한다
-

이번 주 핵심 원칙

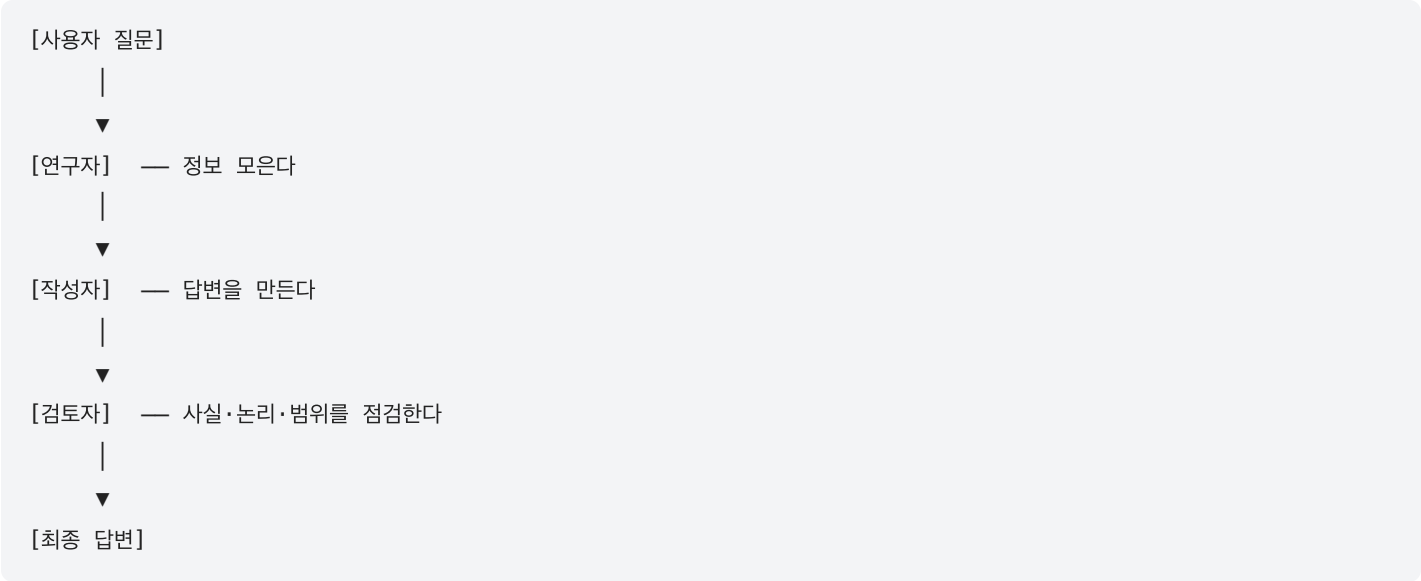
- 프레임워크 비교에 시간을 쓰지 않는다 — LangGraph 한 가지로만 실습한다
 - “멀티에이전트가 멋있어서” 쓰는 것이 아니라 “단일 에이전트로는 풀리지 않아서” 쓰는 감각을 만든다
 - 이번 주에는 구현보다 설계 판단에 집중한다
-

9.1 왜 멀티에이전트인가

하나의 에이전트가 모든 일을 맡으면 다음 문제가 생긴다.

문제	어떻게 드러나는가
컨텍스트 비대	한 대화에 계획·구현·테스트·문서가 모두 섞인다. 모델이 중요한 제약을 놓친다
역할 혼재	검토자처럼 굴어야 할 순간에 작성자처럼 답한다
검증 약화	자기 산출물을 자기가 검토하므로 같은 실수를 반복한다
추적 불가	어디서 잘못됐는지 대화 흐름에서만 확인할 수 있다

멀티에이전트의 기본 아이디어: 일을 책임 단위로 쪼개고, 각 에이전트가 자기 책임만 맡게 한다.



다만 역할을 나누면 **조정 비용**이 든다. 작은 함수 하나 고치는 일에는 멀티에이전트가 오히려 느리고 번거롭다.

9.2 멀티에이전트, 좋은가 나쁜가 (2025년 두 입장)

입장	누가	핵심 한 줄
찬성	Anthropic	일을 나눠서 시키니 단일 에이전트보다 훨씬 잘하더라
신중	Cognition (Devin)	선불리 나누면 에이전트끼리 가정이 안 맞아 더 헤맨다

멀티에이전트는 대립이 아니라 조건의 차이다.

잘 맞는 작업	잘 안 맞는 작업
일을 여러 부분으로 나눌 수 있다	하나의 흐름으로 판단해야 한다
각 부분의 입력과 결과가 분명하다	단계 사이의 가정이 말로 드러나 있지 않다
동시에 진행하면 시간이 줄어든다	순서대로 진행하고 검증만 더하면 충분하다

판단 기준은 간단하다.

이 작업을 서로 독립된 입출력을 가진 작은 작업 2개 이상으로 나눌 수 있는가?

9.3 단일 에이전트와 멀티에이전트 비교

항목	단일 에이전트	멀티에이전트
구조	단순	복잡
구현 비용	낮음	높음
역할 분리	약함	강함
검증 루프	사용자가 챙겨야 함	검토 역할을 별도로 둘 수 있음
추적 가능성	대화 흐름에 의존	핸드오프·로그·트레이스로 남음
적합한 작업	작은 수정, 단순 자동화	조사 + 작성 + 검토, 데이터 + 분석 + 보고

9.4 4대 패턴

멀티에이전트 패턴은 우선 다음 네 가지로 정리한다. 실제 프로젝트의 90%는 이 안에서 해결된다.

패턴 1. 순차형 (Sequential)

가장 단순하다. 한 에이전트의 출력이 다음 에이전트의 입력이 된다.

[연구자] → [작성자] → [검토자]

- 장점: 흐름이 명확하다, 디버깅이 쉽다
- 단점: 병렬 처리가 안 된다
- 처음 설계할 때 가장 안전한 출발점

실제 사례:

사례	흐름
논문 요약	논문 읽기 → 핵심 주장 정리 → 한계 검토 → 요약문 작성
학과 공지 정리	공지에서 사실 추출 → 학생 유형별 분류 → 안내문 작성 → 검토
코드 리뷰 보조	변경 파일 파악 → 위험 지점 분석 → 리뷰 코멘트 작성

잘 맞는 조건: - 순서가 자연스럽게 정해져 있다 - 앞 단계 결과가 다음 단계의 입력으로 바로 쓰인다 - 중간 결과를 확인하기 쉽다

잘 안 맞는 조건: - 여러 자료를 동시에 조사해야 한다 - 다음 역할을 상황에 따라 바꿔야 한다 - 실패했을 때 앞 단계로 되돌아가는 흐름이 많다

패턴 2. 계층형 (Hierarchical / Supervisor / Orchestrator)

관리자(Supervisor) 에이전트가 작업 전체를 보고, 하위 에이전트에게 작업을 나눠 준다.

용어 안내: 이 패턴의 *관리자 역할*은 문헌마다 이름이 다르다. - **Supervisor** — LangGraph 공식 용어 (langgraph-supervisor 패키지, “Agent Supervisor” 튜토리얼) - **Orchestrator** — Anthropic 「How we built our multi-agent research system」(2025)의 *orchestrator-worker pattern*, Microsoft Magentic-One - **Manager / Lead Agent** — CrewAI hierarchical process, AutoGen

같은 개념, 다른 이름이다. 본 강의는 LangGraph 코드와 매칭되는 **Supervisor**를 기본 표기로 쓰되, 개념 설명에서는 두 용어를 병기한다.

[Supervisor / Orchestrator]

/ | \

[연구자] [작성자] [검토자]

- 장점: 작업 분배가 동적이다, 실패 시 다시 다른 에이전트로 돌릴 수 있다
- 단점: Supervisor 설계가 잘못되면 무한 루프
- LangGraph의 표준 패턴 (10주차 실습)

실제 사례:

사례	Supervisor의 결정
복잡한 질의응답	자료가 부족하면 연구자에게, 충분하면 작성자에게 보냄
고객 문의 처리	환불 문의는 정책 확인 에이전트, 기술 오류는 진단 에이전트로 보냄
코딩 에이전트	계획이 필요하면 계획자, 코드 수정은 구현자, 실패 테스트는 디버거에게 보냄

잘 맞는 조건: - 작업의 다음 단계가 처음부터 고정되어 있지 않다 - 중간 결과에 따라 다시 조사하거나 다시 작성해야 한다 - 실패했을 때 다른 역할로 되돌릴 필요가 있다

잘 안 맞는 조건: - 항상 같은 순서로 처리하면 충분하다 - Supervisor가 판단할 정보가 state에 제대로 남지 않는다 - 종료 조건이 불분명하다

패턴 3. 협력형 (Cooperative / Parallel)

여러 에이전트가 같은 목표를 향해 병렬로 일한다.

```

[연구자A]  ↙
[연구자B]  └─→ [통합자]
[연구자C]  ↘
  
```

- 장점: 빠르다, Anthropic 사례에서 강한 패턴
- 단점: 결과 충돌·중복 처리 필요

실제 사례:

사례	병렬 작업
시장 조사	가격 조사, 경쟁사 조사, 사용자 리뷰 조사를 동시에 진행
문헌 조사	논문 A, 논문 B, 논문 C를 각각 다른 에이전트가 읽음
여행 계획	항공권, 숙소, 일정, 예산을 동시에 조사

잘 맞는 조건: - 하위 작업들이 서로 독립적이다 - 각 작업의 입력과 출력이 분명하다 - 병렬로 처리하면 시간이 실제로 줄어든다

잘 안 맞는 조건: - 한 단계의 결정이 다음 단계의 전제가 된다 - 여러 에이전트가 같은 자료를 반복해서 읽는다 - 통합자가 충돌하는 결과를 정리할 기준이 없다

패턴 4. 시장형 (Market / Auction)

여러 에이전트가 입찰처럼 작업을 가져간다. 이 패턴은 구현보다 개념 이해에 초점을 둔다.

- 사용 예: 도구가 많고, 어떤 도구가 잘할지 미리 모를 때
- 단점: 구현 복잡도가 매우 높음

기본 아이디어:

[작업 공고]

↓

[에이전트 A: 내가 잘할 수 있음]

[에이전트 B: 비용은 낮지만 느림]

[에이전트 C: 정확도는 높지만 비쌘]

↓

[선택자]

↓

[선택된 에이전트 실행]

실제 사례:

사례	입찰 기준
도구 선택 에이전트	검색, 계산기, DB, 브라우저 중 어떤 도구가 적합한가
번역/요약 파이프라인	빠른 모델, 정확한 모델, 긴 문서 모델 중 무엇을 쓸 것인가
업무 자동화	이메일 에이전트, 캘린더 에이전트, 문서 에이전트 중 누가 맡을 것인가

잘 맞는 조건: - 사용할 수 있는 도구나 에이전트가 많다 - 어떤 에이전트가 잘할지 미리 고정하기 어렵다 - 비용, 속도, 정확도 중 무엇을 우선할지 비교해야 한다

잘 안 맞는 조건: - 에이전트의 능력을 평가할 기준이 없다 - 입찰과 선택 과정이 실제 작업보다 더 오래 걸린다 - 작은 과제에 과한 구조가 된다

패턴 비교 요약

패턴	핵심 질문	대표 사례
순차형	이 작업은 자연스러운 순서가 있는가	조사 → 작성 → 검토
계층형	다음 역할을 상황에 따라 골라야 하는가	Supervisor / Orchestrator 라우팅
협력형	독립 하위 작업을 동시에 처리할 수 있는가	여러 문서 병렬 조사
시장형	여러 에이전트 중 누가 맡을지 선택해야 하는가	도구·모델 선택

9.5 핸드오프와 공유 메모리

핸드오프(Handoff)

한 에이전트가 “이건 내 일이 아니다, 다음 에이전트가 처리해라”라고 넘기는 행위다.

LangGraph에서는 `Command` 객체로 표현한다.

```
from langgraph.types import Command

def researcher(state):
    # ... 조사 작업
    return Command(
        goto="writer",          # 다음 에이전트로 넘김
        update={"research_notes": notes} # 상태에 결과 추가
    )
```

핵심은 다음 에이전트가 받을 수 있는 형태로 결과를 정리해 두는 것이다.

공유 메모리

에이전트들이 같이 보는 상태(State)다. LangGraph에서는 `TypedDict` 로 정의한다.

```
from typing import TypedDict

class TeamState(TypedDict):
    question: str          # 사용자 질문
    research_notes: str     # 연구자 산출물
    draft: str             # 작성자 산출물
    review: str            # 검토자 산출물
    final: str             # 최종 답변
```

자주 하는 실수는 모든 에이전트가 모든 필드를 다 본다고 생각하는 것이다. 실제로는 각 에이전트가 자신에게 필요한 필드만 읽어야 컨텍스트가 깨끗해진다.

9.6 멀티에이전트 평가: 무엇을 보는가

세 벤치마크는 이름만 알아 둔다.

벤치마크	한 줄
GAIA	도구를 조합해야 풀리는 일반 질문
AgentBench	웹·DB·OS 등 8개 환경에서 다단계 추론
WebArena	실제 웹 사이트에서 작업 완료율

평가는 먼저 다음 4가지 기준으로 시작한다.

1. 작업 완료율 — 의도한 산출물이 나왔는가
2. 근거 — 왜 그렇게 답했는지 설명할 수 있는가
3. 실패 인지 — 모르는 것을 “모른다”고 답하는가
4. 사람 개입 횟수 — 끝까지 가기 위해 몇 번 손댔는가

9.7 언제 멀티에이전트를 쓰지 말아야 하는가

이번 주에서 가장 중요한 부분이다. 다음에 해당하면 단일 에이전트가 낫다.

- 작업이 5분 안에 끝나는 작은 수정이다
- 변경 파일이 1개이고 테스트도 단순하다
- 역할을 나누어도 입출력이 분명해지지 않는다
- 한 에이전트의 답을 다른 에이전트가 그대로 옮겨 쓰게 된다 (역할이 같다는 신호)
- 도구 권한을 통제하기 어렵다

“멀티에이전트로 만들었다”는 자체가 가치가 아니다. **단일 에이전트로 안 풀리는 이유**를 한 문장으로 말할 수 없다면, 단일 에이전트로 시작한다.

9.8 수업 실습: 예시 과제를 함께 설계하기

실습 시나리오

이번 주에는 코드를 먼저 만들지 않는다. 하나의 예시 과제를 정해 설계 과정을 먼저 살펴보고, 같은 흐름을 따라 직접 작성한다.

목표는 “에이전트를 여러 개 쓰는 것”이 아니라, **작업을 역할·입력·출력·검증 기준으로 나눌 수 있는지를 판단하는** 것이다.

수업 예시 과제:

학과 공지 여러 개를 읽고, 학생 유형별로 해야 할 일을 정리한다.

작업 위치

9~13주차 실습과 과제는 주차별 `practice/chapter*/code` 폴더를 따로 만들지 않고, 저장소 루트의 `multi-agent/` 하나에서 이어서 진행한다. Python 가상환경도 저장소 루트에 한 번만 만든다.

macOS / Linux

```
# 프로젝트 루트에서
python3 -m venv .venv
source .venv/bin/activate

mkdir -p multi-agent/code multi-agent/docs multi-agent/data/input multi-agent/data/output
cd multi-agent
```

Windows (PowerShell)

```
# 프로젝트 루트에서
python -m venv .venv
.venv\Scripts\Activate.ps1

New-Item -ItemType Directory -Force -Path multi-agent\code, multi-agent\docs, multi-agent\data\input, multi-agent\data\output | Out-Null
cd multi-agent
```

Windows (cmd)

```
REM 프로젝트 루트에서
python -m venv .venv
.venv\Scripts\activate.bat

mkdir multi-agent\code multi-agent\docs multi-agent\data\input multi-agent\data\output
cd multi-agent
```

작성 파일:

multi-agent/docs/week09_inclass_design.md

Step 1. 예시 과제 이해하기

예시는 아래 조건으로 시작한다.

- 도메인: 학과 공지 정리
- 작업: 여러 공지에서 학생 유형별 행동 목록 만들기
- 이 작업이 반복되는 상황: 수강 신청, 장학금, 졸업 요건, 교환학생 신청 공지가 동시에 올라올 때
- 최종 사용자가 원하는 결과: 1학년, 4학년, 복수전공생, 교환학생 준비생별로 해야 할 일과 마감일을 바로 확인한다

이 작업은 단순 요약이 아니다. 공지마다 대상, 마감일, 준비물, 예외 조건이 다르고, 학생 유형별로 필요한 정보가 달라진다.

Step 2. 단일 에이전트로 부족한 이유 쓰기

다음 질문에 답한다.

1. 한 에이전트가 모두 처리하면 어떤 정보가 섞이는가
2. 어떤 검증이 약해지는가
3. 어떤 판단을 사람이 중간에 확인해야 하는가

단일 에이전트로도 충분하다면, 억지로 멀티에이전트로 만들지 않는다. 이 경우에는 “멀티에이전트를 쓰지 않는 설계”를 결론으로 써도 된다.

예시 답안:

1. 공지的事实 추출, 학생 유형 분류, 최종 안내문 작성이 한 응답에 섞인다.
2. 마감일이나 대상 학년을 잘못 읽어도 스스로 검증하지 못할 수 있다.
3. 학생에게 안내하기 전, 중요한 마감일과 신청 조건은 사람이 확인해야 한다.

Step 3. 역할 나누기

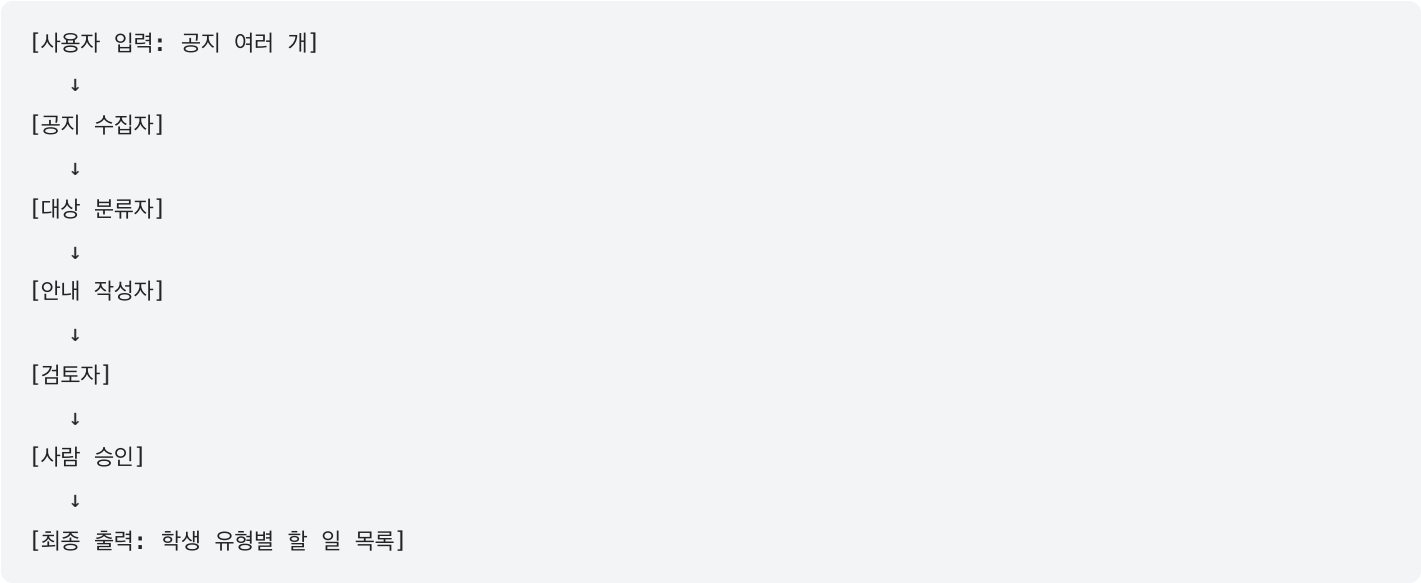
역할은 최소 2개, 최대 4개로 제한한다. 각 역할은 이름이 아니라 **책임**으로 구분한다.

역할	입력	작업	출력	검증 기준
공지 수집자	공지 원문	대상, 마감일, 준비물, 링크 추출	공지별 사실 목록	각 항목에 출처가 있는가
대상 분류자	공지별 사실 목록	학생 유형별 관련 공지 분류	유형별 후보 목록	대상 조건이 맞는가
안내 작성자	유형별 후보 목록	학생이 바로 볼 안내문 작성	유형별 할 일 목록	행동, 마감일, 링크가 포함됐는가
검토자	최종 안내문 + 사실 목록	빠진 조건과 위험문장 확인	수정 요청 또는 승인	원문에 없는 내용이 없는가

역할을 나눴는데 출력이 거의 같다면 잘못 나눈 것이다.

Step 4. 흐름과 상태 설계하기

아래 두 가지를 그리거나 표로 쓴다.



공유 상태도 함께 적는다.

공유 상태 초안

- user_request: 사용자가 원하는 정리 방식
- notice_sources: 공지 원문과 링크
- extracted_facts: 공지별 대상, 마감일, 준비물, 링크
- student_groups: 1학년, 4학년, 복수전공생, 교환학생 준비생
- grouped_tasks: 학생 유형별 후보 작업
- draft_guide: 안내 작성자가 만든 초안
- review_notes: 검토자의 수정 요청
- human_feedback: 사람이 승인하거나 고친 내용
- final_answer: 최종 안내문

모든 항목을 다 쓸 필요는 없다. 자기 설계에 필요한 상태만 남긴다.

Step 5. 사람 승인 지점 정하기

멀티에이전트 설계에는 사람이 확인해야 하는 지점이 있어야 한다. 모든 단계를 자동으로 통과시키면 잘못된 안내가 그대로 나갈 수 있다.

예시 답안:

- 승인 지점: 검토자가 최종 안내문을 확인한 뒤
- 사람이 확인해야 하는 이유: 마감일, 신청 자격, 링크 오류는 학생에게 직접 피해를 줄 수 있다
- 승인 후 다음 단계: 최종 안내문을 학과 홈페이지나 공지 채널에 맞게 다듬는다

Step 6. 실패 조건 쓰기

멀티에이전트 설계는 실패 조건까지 있어야 한다.

예시 답안:

1. 공지가 1개뿐이고 단순 전달이면 단일 에이전트가 더 낫다.
2. 공지 수집자가 마감일을 잘못 추출하면 이후 모든 역할의 결과가 틀어진다.
3. 대상 분류자가 복수전공생과 전과생 조건을 혼동하면 잘못된 안내가 나간다.
4. 검토 없이 자동 게시하면 원문에 없는 조건이 추가될 수 있다.

Step 7. 멀티에이전트를 쓰는 근거 정리하기

마지막으로 “왜 이 작업을 멀티에이전트로 나누는가”를 짧게 정리한다.

예시 답안:

- 사실 추출, 대상 분류, 안내문 작성, 검토의 책임이 서로 다르다.
- 각 단계의 입력과 출력이 표로 분리된다.
- 사람이 승인해야 하는 지점이 분명하다.

Step 8. 5문장 회고 쓰기

예시 답안:

1. 이 작업은 단순 요약보다 대상 분류와 검증이 중요하다.
2. 역할을 나누면 공지 원문에서 뽑은 사실과 최종 안내문을 분리할 수 있다.
3. 가장 중요한 공유 상태는 마감일과 신청 대상이다.
4. 공지가 적거나 조건이 단순하면 멀티에이전트가 오히려 느리다.
5. 10주차 Supervisor 패턴에서는 검토자가 수정 요청을 보낼 때 어느 역할로 되돌릴지 결정할 수 있다.

9.9 Homework: 자기 주제로 설계 과제 만들기

목표

수업시간에 함께 만든 예시를 바탕으로, 각자 별도 주제 하나를 골라 멀티에이전트 설계안을 작성한다.

작성 파일:

multi-agent/docs/week09_homework_design.md

활동 방법

1. 자기 도메인을 한 줄로 정한다 (예: 학과 공지 요약, 논문 검색, 게임 가이드 작성).
2. 그 도메인에서 자주 하는 작업을 1개 고른다.
3. 그 작업을 단일 에이전트로 만들었을 때의 한계를 3줄 적는다.
4. 멀티에이전트로 만든다면 어떤 역할로 나눌지 적는다 (최소 2명).
5. 각 역할의 입력·출력·검증 기준을 적는다.
6. 멀티에이전트를 쓰면 안 되는 조건을 최소 3개 적는다.

좋은 주제 예: - 논문 3편을 비교해 공통 주장과 차이점 정리하기 - 여행 조건을 받아 일정·예산·주의사항 설계하기
- 코드 변경 요청을 읽고 구현 계획·위험·테스트 항목 나누기 - 고객 문의를 분류하고 답변 초안을 작성하기

피해야 할 주제: - 문장 하나 번역하기 - $1 + 1$ 계산하기 - 파일 이름 하나 바꾸기 - 단순 요약 한 번 만들기

자기 점검 기준

- 역할이 책임 단위로 나뉘었는가
- 각 역할의 입력과 출력이 분명한가
- 사람 승인 지점이 필요한 곳에 있는가
- 멀티에이전트를 쓰지 않아도 되는 작업은 아닌가
- 다음 주 Supervisor 패턴으로 옮길 수 있는 흐름인가

설계표

멀티에이전트 설계표

- 도메인:
- 작업:
- 단일 에이전트로 부족한 이유 (3줄):
- 역할 1: 입력 / 작업 / 출력 / 검증 기준
- 역할 2: 입력 / 작업 / 출력 / 검증 기준
- 역할 3 (선택): 입력 / 작업 / 출력 / 검증 기준
- 처리 흐름:
- 공유 상태:
- 사람 승인 지점:
- 멀티에이전트가 더 나아질 것이라 보는 근거:
- 멀티에이전트를 쓰면 안 되는 조건:

이 설계표는 14주차 최종 프로젝트 주제의 출발점이 될 수 있다.

9.10 제출물

필수

- 수업 실습 파일: `multi-agent/docs/week09_inclass_design.md`
- Homework 파일: `multi-agent/docs/week09_homework_design.md`
- Homework 멀티에이전트 설계표 1개
- 역할별 입력.작업.출력.검증 기준
- 처리 흐름 또는 간단한 다이어그램
- 공유 상태 초안
- 실패 조건 3개 이상
- 5문장 회고

회고 질문

1. 이 작업은 왜 단일 에이전트로 부족한가
 2. 역할을 나누었을 때 가장 분명해진 입력과 출력은 무엇인가
 3. 사람 승인이 필요한 지점은 어디인가
 4. 멀티에이전트를 쓰면 오히려 나빠지는 조건은 무엇인가
 5. 다음 주(10주차)의 Supervisor 패턴을 적용한다면 무엇이 달라질 것인가
-

체크리스트

- 단일 에이전트의 한계를 작업 구조 위에서 설명할 수 있다
 - 4대 패턴(순차·계층·협력·시장)을 한 줄씩 설명할 수 있다
 - 핸드오프와 공유 메모리의 차이를 설명할 수 있다
 - 자기 도메인의 작업을 역할·입력·출력·검증 기준으로 나눴다
 - 멀티에이전트가 *불리/한* 조건을 설계안에 포함했다
 - 자기 도메인의 작업으로 멀티에이전트 설계표를 만들었다
-

참고 자료

- Anthropic 「How we built our multi-agent research system」 (2025)
 - Cognition 「Don't build multi-agents」 (2025)
 - LangGraph 1.0 GA 발표: <https://changelog.langchain.com/announcements/langgraph-1-0-is-now-generally-available>
 - LangGraph Multi-Agent 가이드: https://langchain-ai.github.io/langgraph/concepts/multi_agent/
 - GAIA 벤치마크: <https://huggingface.co/gaia-benchmark>
 - Notion ch6 「멀티 에이전트 시스템」
-

다음 주 예고

10주차에는 LangGraph의 **Supervisor** 패턴으로 3-에이전트 시스템을 구현한다. Supervisor가 동적으로 어떤 에이전트에게 일을 줄지 결정하는 흐름과, 체크포인트로 대화를 이어 가는 방법을 다룬다.